



Post-Processing for True Random Bits Generator

Tirumala Reddy B H
Roll No: EE23MT022

Supervisor:
Prof. Naveen Kadayinti
Prof Gopal Krishna Kamath M

Electrical, Electronics and Communication Engineering
Indian Institute of Technology Dharwad

November 14, 2025

Outline

1. Introduction
2. TRNG Building Blocks
3. Post-Processing
4. New Method of Post-Processing
5. Testing the random bits
6. Conclusion

Introduction

Introduction

Random Numbers

Definition

Each value in a defined set has an equal chance of selection, with:

- Unpredictable
- Uniformly distributed
- Independence between values

Types of Random Number Generators

True Random Generators (TRNG)

- From physical phenomena
- Non-deterministic
- Used in cryptography
- Hardware-based

Pseudorandom Generators (PRNG)

- Algorithm-based
- Uses seed value
- Deterministic
- Faster

Example: Dice rolls, coin tosses - equally distributed and unpredictable

Why Do We Need Random Numbers?

Key Applications

- **Cryptography (TRNG)** – Encryption keys, secure tokens, password generation
- **Gaming & Simulations (PRNG)** – Reproducibility for debugging, Monte Carlo methods
- **Machine Learning (PRNG)** – Neural network initialization, dataset shuffling
- **Lottery Systems, Statistical Sampling, Software Testing** – Fairness and unpredictability

TRNG vs. PRNG: When to Use Which?

True Random (TRNG)

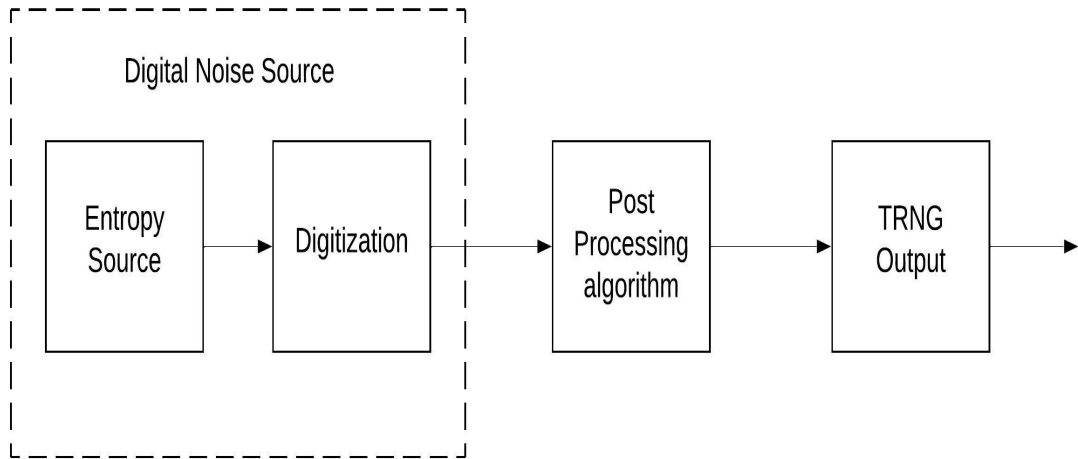
- High security (e.g., cryptography)
- Unpredictable, attack-resistant
- Hardware-based (slower)

Pseudorandom (PRNG)

- Speed and efficiency
- Reproducibility (e.g., simulations)
- Seed-dependent (deterministic)

TRNGs are crucial for security, while PRNGs excel in speed and repeatability. Many systems (e.g., ML, gaming) use PRNGs with TRNG-seeded initialization.

TRNG Building Blocks



References: K. K. e. C, etin Kaya Ko,c (auth.), Cryptographic Engineering. Springer US, 1 ed., 2009.

TRNG Building Blocks

Entropy Source: TRNG 1st Block

The Foundation of True Randomness

- Primary source of raw, unpredictable randomness
- Determines overall TRNG quality and reliability

Physical Phenomena Used

- Thermal noise
- Atomic particle movement
- Jitter noise
- Circuit noise variations

Key Design Considerations

- Must harvest randomness without entropy loss
- Continuous development of new extraction methods
- Critical to maximise entropy from circuit designs

TRNG Building Blocks

Digitization: TRNG 2nd Block

Analog-to-Digital Conversion

- Converts continuous analog signals to discrete bits (0/1)
- Bridges the physical world with digital systems

Implementation Approaches

- Sometimes integrated with an entropy source
- May eliminate the need for a separate ADC in some designs

Critical Challenges

- Voltage threshold stability (temperature sensitivity)
- Typically uses only LSB bits
- Speed vs. entropy preservation trade-off
- Circuit design must minimize entropy waste

Optimal digitization maintains entropy while enabling digital processing

Post-Processing

Post-Processing

3rd Block Post-Processing in TRNGs

Purpose

- Removes bias and correlation from raw bits
- Ensures uniform distribution and independence
- Meets security/statistical standards

Implementation Methods

Simple Techniques

- XOR operations
- Von Neumann corrector

Advanced Techniques

- Cryptographic hash functions
- Resilient processing

Key Requirements

- Low power consumption
- Simple hardware implementation
- Effective bias removal

Project Focus: Developing new post-processing method to enhance randomness quality

Post-Processing

XOR Post-Processing

Working Principle

- Parallel connection of TRNG circuits
- Averages out fluctuations
- Simple digital implementation

Advantages

- Averages the bias
- Simple to implement

TRNG 1	TRNG 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Table: XOR Truth Table

Limitations

- Data-dependent (single TRNG failure critical)
- Easy to implement, but needs an extra TRNG
- Only reduces bias, doesn't eliminate

References: S. T. Chandrasekaran, V. E. G. Karnam, and A. Sanyal, "0.36-mw, 52-mbps true random number generator based on a stochastic delta-sigma modulator," IEEE Solid-State Circuits Letters, vol. 3, pp. 190–193, 2020.

Post-Processing

Von Neumann Post-Processing

Algorithm

- Divides bits into 2-bit non-overlapping blocks
- Keeps only 01→0 and 10→1 mappings
- Discards 00 and 11 completely

Advantages

- Perfect bias removal (ideal conditions)
- Simple to implement

Bit 1	Bit 2	Output
0	0	-
0	1	0
1	0	1
1	1	-

Table: Von Neumann Processing

Drawbacks

- 75% data loss (low throughput)
- Fails with correlated bits
- Variable output rate

References: C. Li, Q. Wang, J. Jiang, and N. Guan, "A metastability-based true random number generator on fpga," in 2017 IEEE 12th international conference on ASIC (ASICON), pp. 738–741, IEEE, 2017.

Post-Processing

LFSR Post-Processing

Operation

- Shift register with XOR feedback
- Spreads entropy across bits
- Maximum period: $2^n - 1$ cycles

Benefits

- Reduces bias
- FPGA/ASIC friendly
- Configurable polynomial

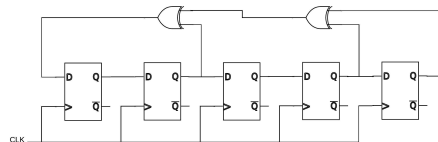


Figure: 5-bit LFSR Example

Considerations

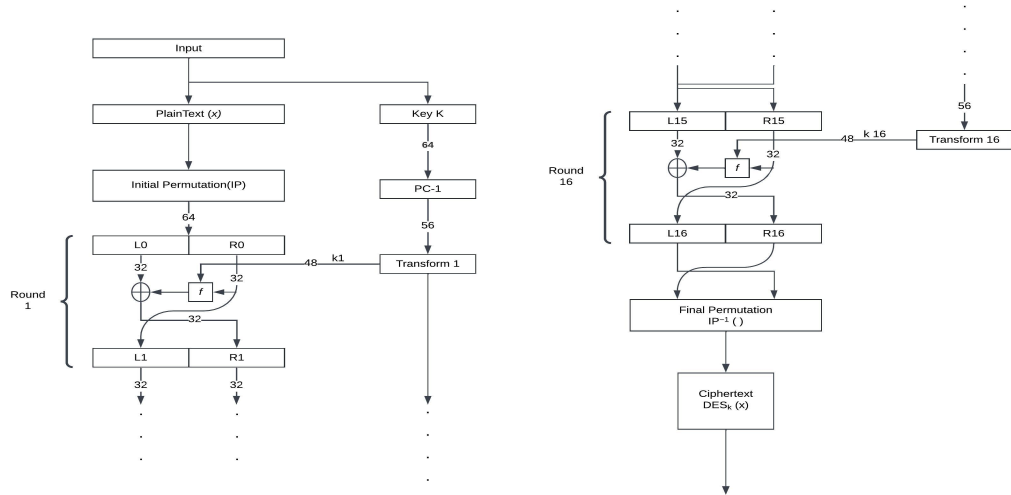
- Not cryptographically secure
- Requires proper initialization
- Feedback taps affect quality

References: X. Wu, Y. Ma, T. Chen, and N. Lv, "A distinguisher for RNGs with LFSR post-processing," in Information Security and Cryptology (Y. Wu and M. Yung, eds.), (Cham), pp. 328–343, Springer International Publishing, 2021.

Post-Processing

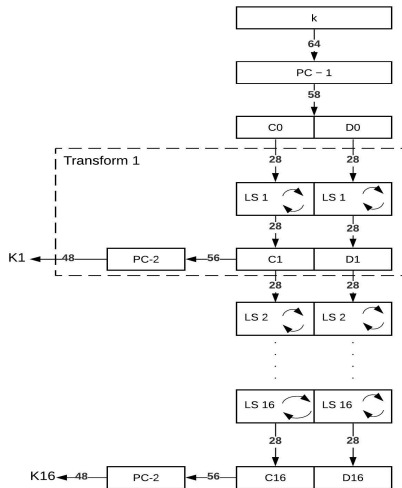
Data Encryption Standard

DES Working Principle



References: C. Paar, Understanding Cryptography: A Textbook for Students and Practitioners. Springer, 2010 ed., 2014.

Key Schedule: Subkey Generation



References: C. Paar, Understanding Cryptography: A Textbook for Students and Practitioners. Springer, 2010 ed., 2014.

DES S-Box Structure

S-Box S1 (Example)

Input (b1b6)	Middle 4 bits (b2b3b4b5)							
	0	1	2	3	4	5	6	7
00	14	4	13	1	2	15	11	8
01	0	10	6	12	5	9	0	7
10	15	2	8	14	12	0	1	10
11	7	11	4	1	9	12	14	2

- 6-bit input: **b1b2b3b4b5b6**
- **b1b6** selects row (00-11)
- **b2b3b4b5** selects column (0-15)

Lookup Example

Input: 011001

- 1 Split: 011001
- 2 Row bits: 01 (01)
- 3 Column bits: 1001 (9)
- 4 Output: Value at row 1, column 9 = 6 (0110)

Key Features

- 8 unique S-Boxes in DES
- Each maps 6 bits $\rightarrow$ 4 bits
- Fixed substitution tables

S-Box Design Properties

Cryptographic Properties

- **Non-Linearity:** No linear input-output relationships
- **Completeness:** All output bits depend on all input bits
- **Balance:** Outputs uniformly distributed
- **Bit Diffusion:** ≥ 2 output bits change for single input bit change

Security Purpose

- Primary source of confusion in DES
- Resists linear cryptanalysis
- Thwarts differential cryptanalysis
- Provides non-linear substitution

Differential Properties

- Changing middle bits changes ≥ 2 output bits
- Changing edge bits changes the output
- Limits collisions to adjacent S-Boxes

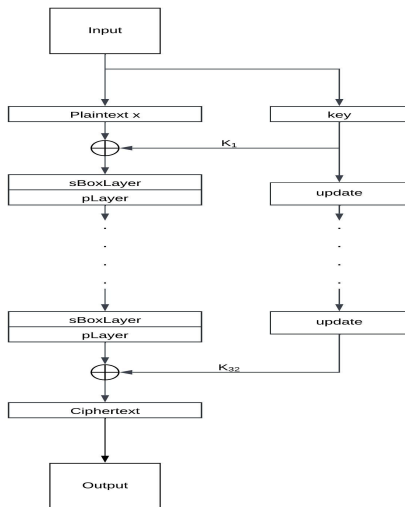
Design Constraints

- No output bit should mimic input bits
- Outputs must appear random
- Carefully designed to resist known attacks

Post-Processing

PRESENT cipher

PRESENT Working Principle



References: C. Paar, Understanding Cryptography: A Textbook for Students and Practitioners. Springer, 2010 ed., 2014.

Substitution Layer (sBoxLayer)

Core Function

- Introduces **non-linearity** to the cipher
- PRESENT uses a **4-bit S-box** (nibble substitution)
- Each 4-bit state nibble is replaced via a fixed lookup table

S-Box Example

Input	0	1	2	3	4	5	6	7
Output	C	5	6	B	9	0	A	D

Example: Input 0101 (5) → Output 0000 (0)

Cryptographic Properties

- **Balanced outputs:** Equal probability for all nibbles
- **Non-linearity:** No linear input-output relationships
- **Resistance:** Thwarts linear/differential cryptanalysis

Permutation Layer (pLayer)

Diffusion Mechanism

- Shuffles state bits via **fixed permutation pattern**
- Breaks direct input-output relationships
- Spreads local changes through the entire block

Permutation Example

Bit Position	New Position
0	0
1	16
2	32
3	48
...	...
63	63

Security Impact

- **Avalanche effect:** Small changes affect entire state
- Prevents pattern recognition
- Complements sBoxLayer's non-linearity

Design Principle

"Every output bit should depend on every input bit after a few rounds"

Key Schedule

Round Key Generation

- Derives **round keys** K_i from master key K
- 64-bit round keys extracted from rotated key state
- Unique keys per round via:
 - Bitwise rotations
 - S-box substitutions
 - XOR with round counter

Key Schedule Steps

- 1 Rotate key state left by 61 bits
- 2 Pass leftmost 4 bits through S-box
- 3 XOR bits 15-19 with round counter
- 4 Extract first 64 bits as K_i

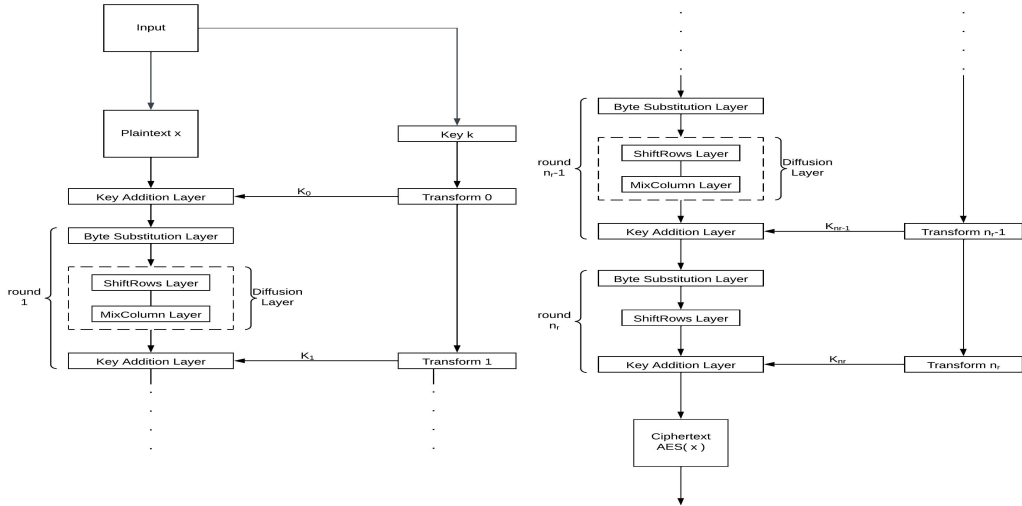
Security Features

- Prevents key recovery attacks
- Ensures key diversity
- Resists related-key attacks

Post-Processing

Advanced Encryption Standard

AES Working Principle



References: C. Paar, Understanding Cryptography: A Textbook for Students and Practitioners. Springer, 2010 ed., 2014.

Byte Substitution Layer (S-Box)

Non-Linear Transformation

- Replaces each state byte using a precomputed lookup table
- Constructed via mathematical operations in $GF(2^8)$
- Provides **confusion** in the cipher

S-Box Example

	Input (hex)			
	0x	1x	2x	3x
0x	63	7C	77	7B
1x	CA	82	C9	7D
2x	B7	FD	93	26
3x	04	C7	23	C3

Cryptographic Properties

- **Non-linear** inverse in $GF(2^8)$
- **Balanced**: 256 unique outputs
- **Fixed-point free**: $S(a) \neq a$

Construction Steps

1. Multiplicative inverse in $GF(2^8)$
2. Affine transformation
3. Constant XOR (0x63)

Diffusion Layer - ShiftRows

Row Shifting Operation

- Cyclically shifts each row of the state matrix
- Different offsets per row (0,1,2,3 for AES)
- Breaks column alignment

Before ShiftRows

$$\begin{bmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{bmatrix}$$

After ShiftRows

$$\begin{bmatrix} A & B & C & D \\ F & G & H & E \\ K & L & I & J \\ P & M & N & O \end{bmatrix}$$

Security Purpose

- Disperses bytes across columns
- Ensures subsequent MixColumns affect the entire state
- Works with S-Box to provide **diffusion**

Diffusion Layer - MixColumns

Column Mixing Operation

- Matrix multiplication in $GF(2^8)$
- Each output byte depends on all 4 bytes in the column
- Uses irreducible polynomial $x^8 + x^4 + x^3 + x + 1$

MixColumns Matrix

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \times \begin{bmatrix} S_{0,c} \\ S_{1,c} \\ S_{2,c} \\ S_{3,c} \end{bmatrix} = \begin{bmatrix} S'_{0,c} \\ S'_{1,c} \\ S'_{2,c} \\ S'_{3,c} \end{bmatrix}$$

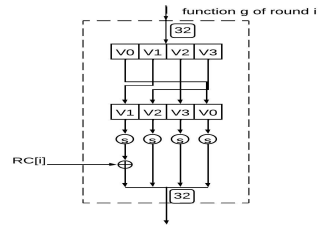
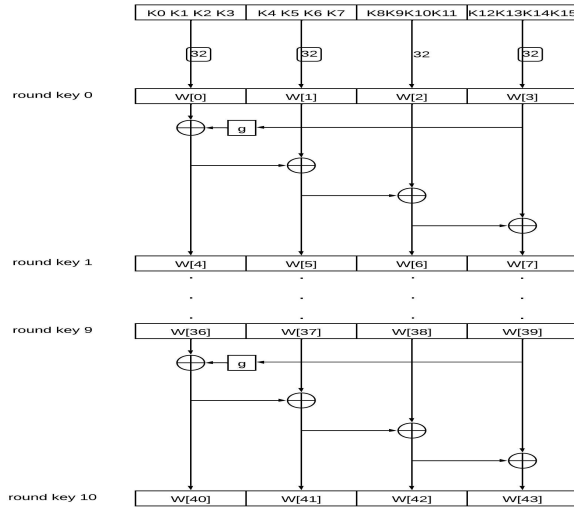
Properties

- **Maximum diffusion:** 1-byte change affects entire column
- **Invertible:** Crucial for decryption
- **Branch number** = 5 (optimal)

$GF(2^8)$ Operations

- Addition = XOR
- Multiplication modulo an irreducible polynomial

Key Schedule



References: C. Paar, Understanding Cryptography: A Textbook for Students and Practitioners. Springer, 2010 ed., 2014.

New Method of Post-Processing

Current Cryptographic Approach

- Improves TRNG security
- Reduces bias and correlation
- Increases implementation complexity
- Creates understanding challenges

Our Proposed Method

- New post-processing approach
- Replaces S-box with barrel shifter
- Uses key as shift amount control

Implementation Focus

- Detailed method implementation
- Different approaches analyzed
- Challenges in implementation

New Method of Post-Processing

Barrel Shifter & XOR Operations

Barrel Shifter Post-Processing Architecture

Working Principle

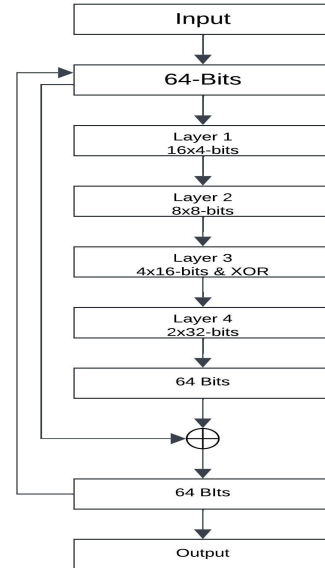
- Input: 64-bit TRNG output blocks
- Hierarchical processing:
 - 4-bit $\rightarrow$ 8-bit $\rightarrow$ 16-bit $\rightarrow$ 32-bit
 - Feistel-like network structure

4-bit Processing Example

- 1 Split: $0xABCD \dots \rightarrow 16 \times 4\text{-bit}$
- 2 Shift: $0b1011 \xrightarrow{ii^2} 0b1100$
- 3 Swap: Random nibble exchange
- 4 Combine: $2 \times 4\text{-bit} \rightarrow 8\text{-bit}$

Key Operations

- Shift amount derived from:
 - 2-3 bits (4-bit)
 - 3-7 bits (8-bit)
 - 5-8 bits (16-bit)



Multi-Level Processing Stages

32-bit Stage Example

0x89ABCDEF →

- Shift left by 5 bits:
0x1579BDE0
- Swap halves:
0xBDE01579
- XOR with neighbor:
 $0xBDE01579 \oplus 0x...$

Iterative Processing

- 2-4 complete iterations
- Final 64-bit recombination
- XOR feedback loop

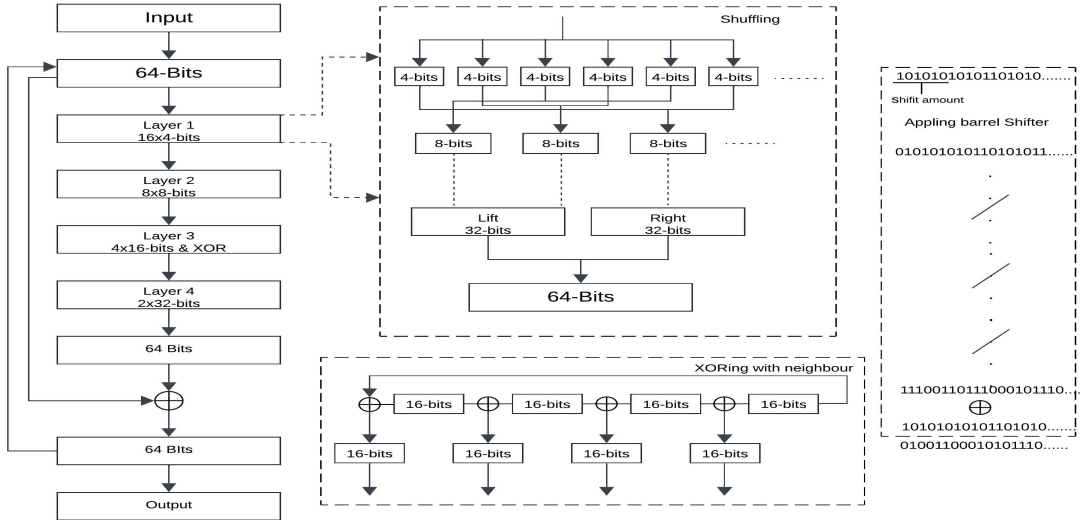
Bit-Wise Transformation

Stage	Operation
4-bit	Bit rotation + swap
8-bit	Bit rotation + cross-swap
16-bit	Bit rotation + XOR + cross-swap
32-bit	Bit rotation + halve swap

Security Properties

- Non-linear bit diffusion
- Key-dependent shifting
- Avalanche effect
- Resistance to pattern analysis

Working Principle



New Method of Post-Processing

Implementation-Comparison

Method	Processing Order	Key Features	Challenges	Observations
Barrel Shifter Only	64→32→16→8→4-bit	• Multi-level barrel shifting • Shift amounts derived from input bits	• High iterations needed • Correlation reduction insufficient	• Correlation reduced but bias remains • High power/execution time
Barrel Shifter + XOR (Final)	64→32→16→8→4-bit	• Adds XOR after recombination • Non-linear feedback	• Still requires many iterations	• Better bias reduction • Still high resource usage
Barrel Shifter + XOR (Neighbor Blocks)	64→32→16→8→4-bit	• XOR with neighboring blocks at each level • Enhanced non-linearity	• Complex swapping logic • Higher hardware overhead	• Stronger correlation/bias reduction • More efficient than previous methods
Barrel Shifter + XOR (4-bit Focus)	64→4→8→16→32-bit	• Starts with 4-bit blocks (most effective) • XOR at 16-bit level	• Optimal shift amount selection critical	• Fastest bias/correlation reduction • Fewer iterations (2-4)

Testing the random bits

Standardized Test Suites

- **NIST SP 800-22** (2001/2010)
 - 15 cryptographic tests
 - FIPS 140 compliance
 - TRNG/PRNG evaluation
- **FIPS 140-2/3** (2001/2019)
 - Continuous RNG tests
 - Mandatory for the government. systems
 - Power-up tests
- **AIS-31** (2011)
 - German BSI standard
 - Hardware TRNG focus
 - PTG.2/P2 class certification

Academic/Industry Tests

- **Diehard/Dieharder** (1995/2003)
 - Marsaglia's original suite
 - Dieharder adds 18 new tests
 - Chi-square evaluation
- **TestU01** (2007)
 - SmallCrush/Crush/BigCrush
 - 126+ statistical tests
 - PRNG stress testing
- **PractRand** (2014)
 - Terabyte-scale testing
 - Adaptive test batteries
 - Early failure detection

Quick Checks

- **ENT** (1998): Basic entropy/chi-square/mean tests
- **rdrand** verification: On-chip CPU RNG validation

Testing the random bits

NIST Randomness Testing Framework

Core Hypothesis Test

- **Null Hypothesis (H_0):** Sequence is random
- **Alternative (H_a):** Sequence is non-random
- Significance level $\alpha = 0.01$ (99% confidence)

Test Components

- Test statistic computation
- Reference distribution
- Critical value threshold

Error Types

- Type I (α): False positive
- Type II (β): False negative

P-value Interpretation

- $P \geq \alpha$: Accept randomness
- $P < \alpha$: Reject randomness

References: A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, J. Dray, and S. Vo, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," Tech. Rep.SP 800-22 Rev. 1a, National Institute of Standards and Technology (NIST), 2010.

Randomness Testing Assumptions

Fundamental Requirements

- **Uniformity:** $P(0) = P(1) = 0.5$
- **Scalability:** Subsequences maintain randomness
- **Consistency:** Stable across different seeds

Important Notes

- Some failures are expected in truly random sequences
- Requires multiple test runs for reliable assessment
- Tester must interpret results contextually

Practical Implications

- Tests detect deviations from ideal randomness
- Cannot prove randomness absolutely
- $\alpha = 0.01$ balances sensitivity/specificity

Conclusion

Conclusion

Summary

Core Components

- **Entropy Source:**
 - Electronic noise
 - Quantum effects
 - Radioactive decay
- **Digitization:**
 - Analog-to-digital conversion
 - Noise sampling
- **Post-Processing:**
 - Bias removal
 - Entropy amplification

Our Novel Contribution

- Barrel shifter + XOR post-processing
- Replaces complex cryptographic functions
- Maintains security with lower complexity

NIST Validation (SP 800-22)

- Frequency Test (uniformity)
- Runs Test (pattern detection)
- Block Frequency (bit blocks)
- Approximate Entropy
- Cumulative Sums
- Serial/Longest Runs

Key Requirements

- Unpredictability
- High entropy
- Bias-free outputs
- Cryptographic resilience

Conclusion

Results

NIST Statistical Test Results Summary

Test	Crypto (AES/DES/PRES)	BS Only	BS+XOR (Final)	BS+XOR (Neighbor)	BS+XOR (4-bit)
Frequency	✓	×	✓	✓	✓
Block Freq.	✓	×	×	✓	✓
Runs	✓	×	×	✓	✓
Longest Run	✓	×	×	✓	✓
FFT	✓	×	✓	✓	✓
Non-Ovl. Temp.	✓	×	×	✓	✓
Universal	✓	×	×	✓	✓
Linear Comp.	✓	×	✓	✓	✓
Approx. Entropy	✓	×	×	✓	✓
Random Excur.	2 – 3/10	0/10	0/10	2 – 3/10	2 – 3/10

- ✓: Passed all 10 sequences (1×10^6 bits each)
- ×: Failed all, X/10: Problematic
- BS: Barrel Shifter, Crypto: AES/DES/PRESENT all showed identical results
- Significance level $\alpha = 0.01$ for all tests

Key Findings

- Cryptographic methods passed all tests
- BS+XOR (4-bit) performed equivalently to crypto methods
- Random Excursions tests are challenging for all methods

Detailed NIST Test Results

Consistent Failures

- **Barrel Shifter Only:**
 - Failed 12/15 tests
 - Only passed Frequency test
- **Barrel Shifter + XOR (Final):**
 - Failed 8/15 tests
 - Issues with Block Frequency, Runs

Common Challenges

- Random Excursions tests problematic
- 2-3 errors out of 10 sequences
- Edge cases in neighbor block XOR

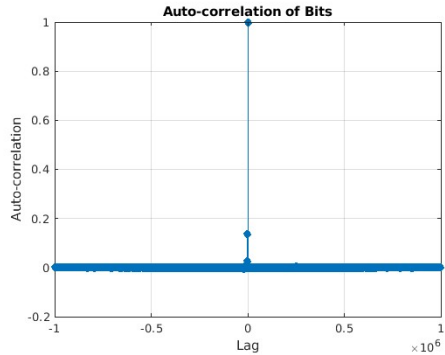
Test Sequence Details

1 million bits per sequence
10 sequences tested per method
 $\alpha = 0.01$ significance level

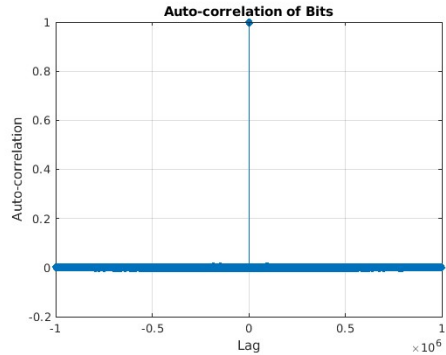
Conclusion

Autocorrelation Results

DES

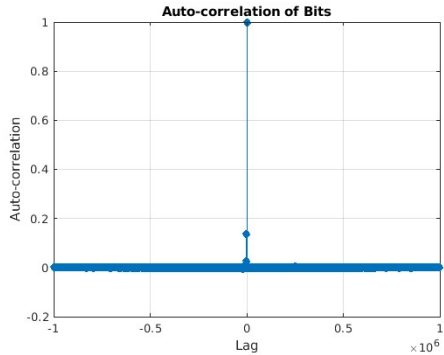


Before Post-Processing

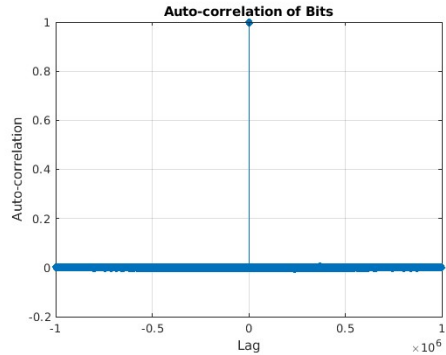


After DES Processing

PRESENT cipher

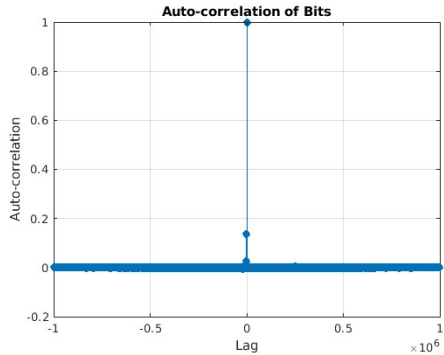


Before Post-Processing

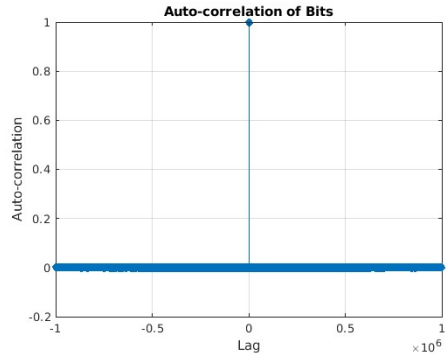


After PRESENT Processing

AES

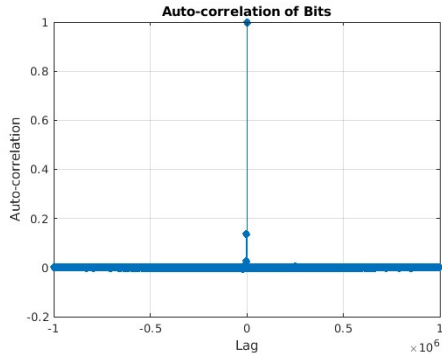


Before Post-Processing

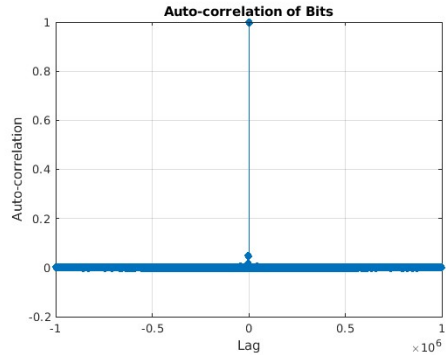


After AES Processing

Barrel Shifter Only

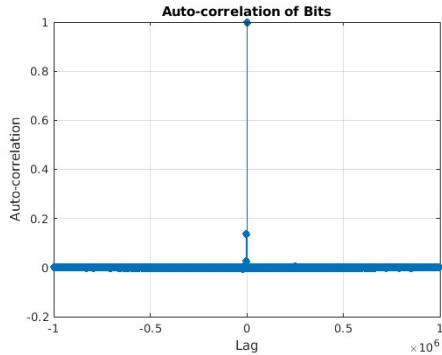


Before Post-Processing

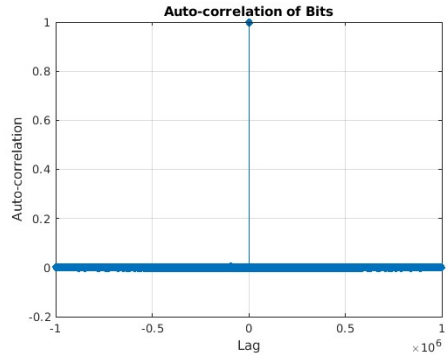


After BS Only Processing

Barrel Shifter + XOR (Final)

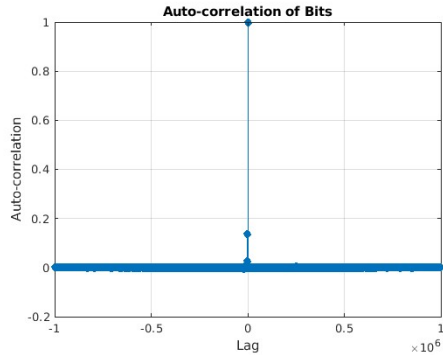


Before Post-Processing

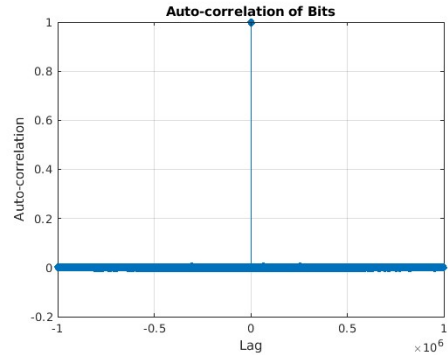


After BS + XOR (Final) Processing

Barrel Shifter + XOR (Neighbour Blocks)

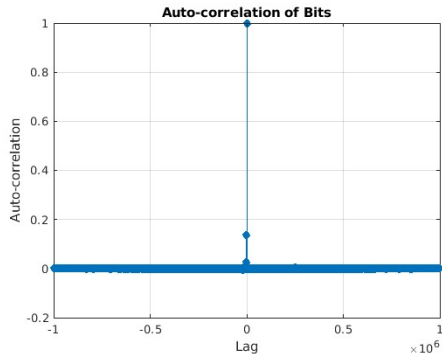


Before Post-Processing

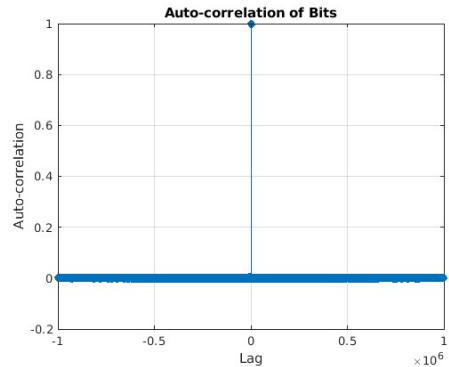


After BS + XOR (NB) Processing

Barrel Shifter + XOR (4-bit Focus)



Before Post-Processing



After BS + XOR (4-bit Focus) Processing

Conclusion

Conclusion & Future Work

Conclusion

- **Success:** Barrel Shifter+XOR/DES/PRESENT pipeline passed all 15 NIST tests
 - Eliminated biases/autocorrelation/entropy deficits
 - Lower overhead than standard crypto

Future Work

- Architecture comparisons
- Bias/entropy quantification
- Hardware-efficient shuffling + nonlinear mixing
- Adaptive throughput optimization

Applications: Cryptographic key generation, secure protocols

Reproducible paradigm for certifiable randomness in high-security systems

Thank You

1st & 2nd Blocks of TRNGs

Amplified Noise-Based TRNG

Design Principle

- Amplifies small voltage fluctuations (thermal noise)
- Uses a comparator to produce digital signals
- Requires significant power for noise amplification

Recent Improvements

- Si nanodevices for high-amplitude noise
- SiN MOSFET for reduced circuit size
- Soft breakdown MOSFET technique

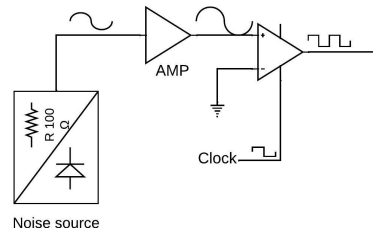


Figure: Amplified Noise TRNG Circuit

Challenges

- Sensitive to temperature fluctuations
- Produces correlated/biased bits
- Complex amplifier design

Oscillator-Based TRNG

Design Principle

- Uses ring oscillator phase jitter
- Odd number of inverters in loop
- Popular in FPGA implementations

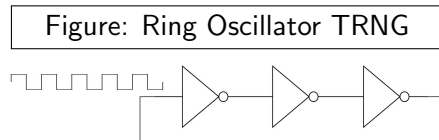
Characteristics

- Simple digital implementation
- Unpredictability grows with more inverters
- Requires post-processing

Challenges

- Non-idealities and bias
- Sensitive to environmental conditions

References:



Metastability-Based TRNG

Design Principle

- Uses cross-coupled inverters
- Nodes settle into metastable states
- Noise determines the final state

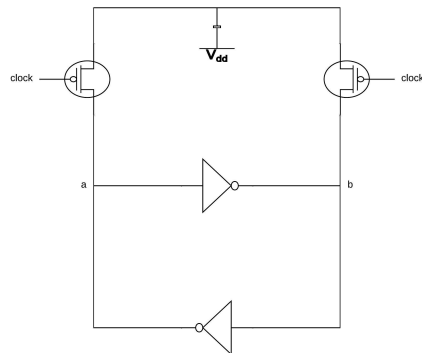
Operation

- Clock transition triggers metastability
- Thermal/power noise breaks symmetry
- Produces random bit output

Challenges

- Sensitive to PVT variations
- Difficult to maintain metastability

Figure: Cross-Coupled Inverters



Chaotic TRNGs

Types

- Continuous-time (analog circuits)
- Discrete-time (digital circuits)

Advantages

- Large bandwidth
- High sensitivity
- Digital implementation is possible

Challenges

- Continuous-time: large area/power
- Discrete-time: finite precision issues

References:

Sigma-Delta Modulator TRNG

Components

- Low-resolution ADC
- DAC for feedback
- Loop filter

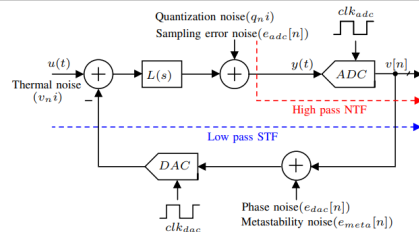
Advantages

- Compact design
- Dual function (TRNG + ADC)
- Multiple noise sources

Challenges

- Analog circuit complexity
- Environmental sensitivity
- Needs post-processing

Figure: $\Sigma - \Delta$ Modulator



NIST 15 Test Suites

Frequency (Monobit) Test

Purpose

Tests if the proportion of 0s and 1s matches the expected 50/50 distribution

Example Bit Distribution

Key Characteristics

- Fundamental test - all others depend on it
- Checks global bit balance
- Detects significant bias

Implementation Notes

- Uses normalized difference statistic
- Critical for initial assessment

Frequency Test within a Block

Purpose

Verifies uniform 1s distribution in M-bit blocks

Parameters

- Block size M (typically 128)
- Expected proportion: $M/2$ ones
- Generalizes the monobit test

Special Case

When $M=1$, it becomes a monobit test

Block	1s Count
1	64
2	62
...	...

Table: Sample Block Counts

Runs Test

Purpose

Analyzes oscillations between 0s and 1s

Run Length Distribution

What it Measures

- Total number of runs
- Lengths of runs
- Distribution of runs

What it Detects

- Too fast oscillation
- Too slow oscillation

Longest Run of Ones Test

Purpose

Assess the longest 1s run consistency

Key Features

- Uses fixed block size ($M=10000$)
- Compares to the expected distribution
- Implicitly tests the longest 0s run

Block	Longest Run
1	12
2	14
...	...

Table: Sample Results

Note

Only tests ones explicitly - zeros follow the same pattern

Binary Matrix Rank Test

Purpose

Checks for linear dependencies

Matrix Rank Example

Methodology

- Divides the sequence into matrices
- Computes ranks
- Compares the rank distribution

Significance

- Detects linear patterns
- Also in the DIEHARD suite

Discrete Fourier Transform (Spectral) Test

Purpose

Detects periodic patterns via frequency analysis

DFT Peak Analysis

Methodology

- Applies DFT to a sequence
- Analyzes peak heights
- 95% threshold comparison

Key Insight

- Expect only 5% peaks above threshold
- Detects repetitive patterns

Non-overlapping Template Matching Test

Purpose

Finds excessive occurrences of m-bit patterns

Search Algorithm

- Slides m-bit window
- Skips to next position after match
- Tests multiple templates

Pattern	Count
101	23
110	19

Table: Template Counts

Note

Specifically targets aperiodic patterns

Overlapping Template Matching Test

Purpose

Detects clustered pattern occurrences

Overlapping Search

Key Difference

- Slides 1 bit after match
- More sensitive to clustering
- Tests a single specified pattern

Comparison

More intensive than the non-overlapping version

Maurer's Universal Statistical Test

Purpose

Measures sequence compressibility

Operation

- Tracks distances between patterns
- Computes compression metric
- Random sequences incompressible

Pattern	Distance
101	15
101	8

Table: Pattern Distances

Interpretation

Low compression = more random

Linear Complexity Test

Purpose

Assesses sequence complexity via LFSR

LFSR Analysis

Key Metric

- Minimum LFSR length needed
- Random sequences need long LFSRs
- Berlekamp-Massey algorithm

Failure Mode

Short LFSR = predictable sequence

Serial Test

Purpose

Checks the m -bit pattern uniformity

Features

- Tests all $2^m m$ – *bitpatterns*
- Overlapping windows
- Generalizes frequency test

Pattern	Count
00	251
01	249
10	253
11	247

Table: $m=2$ Pattern Counts

Special Case

$m=1$ reduces to the frequency test

Approximate Entropy Test

Purpose

Compares adjacent block length frequencies

Block Length Comparison

Method

- Counts m and $m+1$ bit blocks
- Computes entropy difference
- Random sequences show uniformity

Relation

Similar to a serial test, but with a different metric

Cumulative Sums (Cusum) Test

Purpose

Analyzes random walk maximal excursions

Random Walk Plot

Transformation

- $0 \rightarrow -1, 1 \rightarrow +1$ mapping
- Computes partial sums
- Tracks maximal deviation

Interpretation

Large excursions indicate non-randomness

Random Excursions Test

Purpose

Analyzes random walk state visits

Features

- 8 specific states (± 1 to ± 4)
- Counts complete cycles
- Multiple hypothesis tests

State	Visits
+1	12
-2	8

Table: State Visits

Note

Focuses on cycles returning to the origin

Random Excursions Variant Test

Purpose

Extended state visit analysis

State Transition Diagram

Differences

- 18 states (± 1 to ± 9)
- Counts all visits (not just cycles)
- More sensitive detection

Application

Detects more subtle deviations